

08_multifile - Student and Course Management System

This is a complete multi-file C++ project demonstrating classes, objects, and multi-file organization.

Tutorial tasks

1. make a multi-file c++ project with the following files:
 - student.h
 - student.cpp
 - course.h
 - course.cpp
 - main.cpp
2. Create a `Professor` class and assign professors to courses
 - i. add professor.h and professor.cpp to the project
 - ii. in professor.h, declare the `Professor` class with member variables (id, name, courses) and member functions (print, addCourse, removeCourse)
 - iii. in professor.cpp, implement all `Professor` class member functions
 - iv. in main.cpp, create a `Professor` object and assign it to a course using the `addCourse` function, print the professor's name and the courses they are assigned to using the `print` function.

Project Structure

```
08_multifile/
├─ student.h      # Student class declaration
├─ student.cpp    # Student class implementation
├─ course.h       # Course class declaration
├─ course.cpp     # Course class implementation
├─ main.cpp       # Main program demonstrating the system
└─ README.md      # This file
```

Files Description

Header Files (.h)

- **student.h**: Declares the `Student` class with member variables (`id`, `name`, `major`, `gpa`) and member functions (`print`, `isHonorStudent`, `getStanding`)
- **course.h**: Declares the `Course` class with member variables (`code`, `title`, `credits`, `capacity`, `enrolledStudents`) and member functions (`initialize`, `enrollStudent`, `print`, `isFull`, `getAvailableSeats`)

Implementation Files (.cpp)

- **student.cpp**: Implements all `Student` class member functions
- **course.cpp**: Implements all `Course` class member functions

Main File

- **main.cpp**: Creates `Student` and `Course` objects, demonstrates enrollment functionality, and prints final status

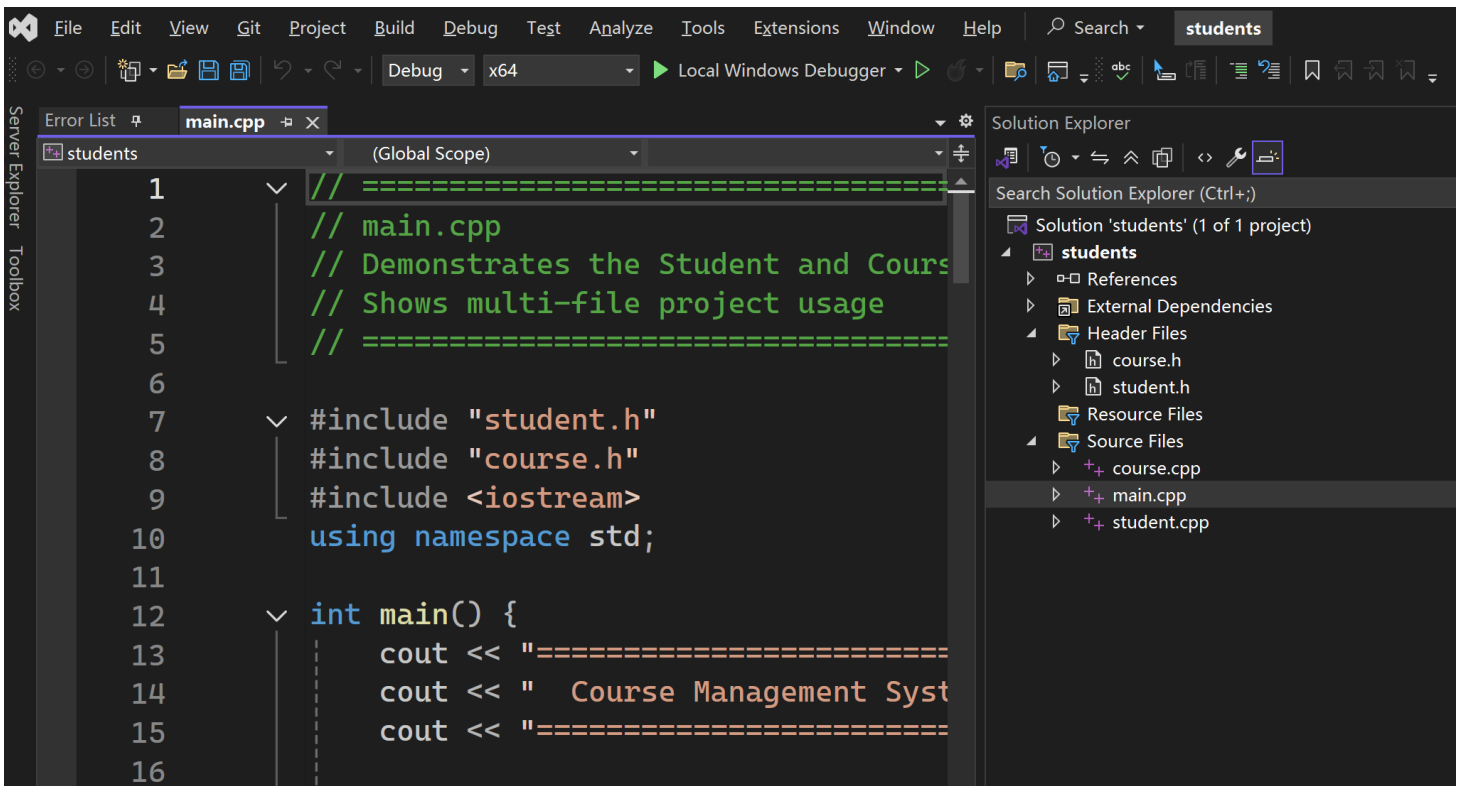
Compilation Instructions

Using Xcode

1. Create a new C++ project
2. Add all files (`student.h`, `student.cpp`, `course.h`, `course.cpp`, `main.cpp`) to the project
3. Build and run (Cmd+R)

Using Visual Studio

1. Create a new C++ Console Application project
2. Add all files to the project using solution explorer
3. Build Solution (Ctrl+Shift+B)
4. Run (F5)



Expected Output

```
=== University Course Management System ===
```

```
--- Enrolling Students ---
```

```
Success: Student 1001 enrolled in CS101
```

```
Success: Student 1002 enrolled in CS101
```

```
Error: Course CS101 is full!
```

```
Success: Student 1001 enrolled in MATH201
```

```
Success: Student 1002 enrolled in MATH201
```

```
--- Course Status ---
```

```
CS101: Intro to Programming
```

```
Credits: 3
```

```
Enrollment: 2/2
```

```
Enrolled Students: 1001, 1002
```

```
MATH201: Calculus II
```

```
Credits: 4
```

```
Enrollment: 2/30
```

```
Enrolled Students: 1001, 1002
```

```
--- Student Status ---
```

```
Alice Chen - Standing: Excellent
```

```
Bob Smith - Standing: Good
```

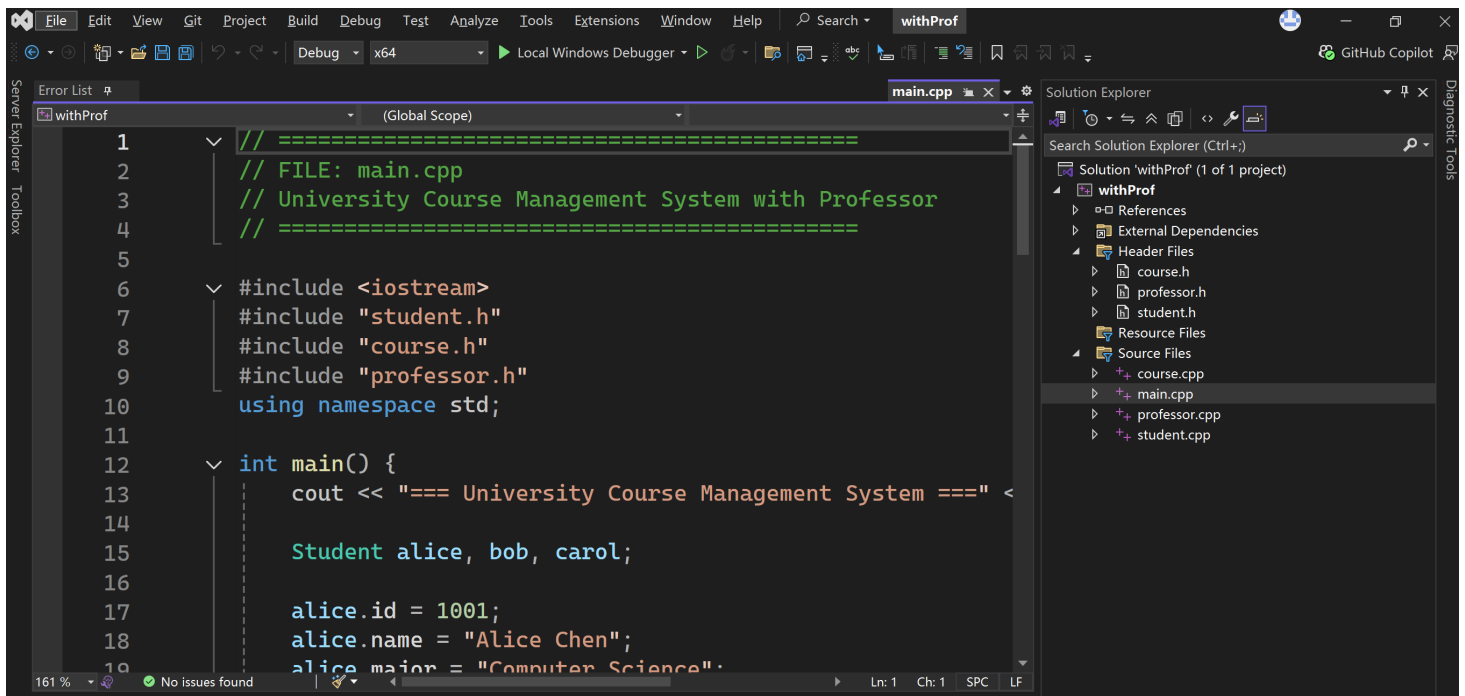
```
Carol White - Standing: Excellent
```

Key Concepts Demonstrated

1. **Multi-file organization:** Header files for declarations, source files for implementations
2. **Header guards:** `#ifndef / #define / #endif` pattern to prevent double inclusion
3. **Class separation:** Student and Course classes in separate files
4. **Class relationships:** Course class stores and uses Student IDs
5. **Member functions:** Using `const` for read-only functions
6. **Scope resolution operator:** `ClassName::functionName` syntax
7. **Array of objects:** Fixed-size array for enrolled students

Add a new class to the project

1. Create a `Professor` class and assign professors to courses
2.
 - i. add `professor.h` and `professor.cpp` to the project
 - ii. in `professor.h`, declare the `Professor` class with member variables (`id`, `name`, `courses`) and member functions (`print`, `addCourse`, `removeCourse`)
 - iii. in `professor.cpp`, implement all `Professor` class member functions
 - iv. in `main.cpp`, include the `professor.h` file and create a `Professor` object and assign it to a course using the `addCourse` function, print the professor's name and the courses they are assigned to using the `print` function.



The screenshot shows the Visual Studio IDE with the `main.cpp` file open. The code in `main.cpp` includes `<iostream>`, `"student.h"`, `"course.h"`, and `"professor.h"`, and uses the `std` namespace. The `main` function prints the title "University Course Management System", creates a `Student` object named `alice` with `id = 1001`, `name = "Alice Chen"`, and `major = "Computer Science"`. The `Solution Explorer` on the right shows the project structure for `withProf`, including `course.h`, `professor.h`, `student.h`, `course.cpp`, `main.cpp`, `professor.cpp`, and `student.cpp`.

```
1 // =====  
2 // FILE: main.cpp  
3 // University Course Management System with Professor  
4 // =====  
5  
6 #include <iostream>  
7 #include "student.h"  
8 #include "course.h"  
9 #include "professor.h"  
10 using namespace std;  
11  
12 int main() {  
13     cout << "=== University Course Management System ===" << endl;  
14  
15     Student alice, bob, carol;  
16  
17     alice.id = 1001;  
18     alice.name = "Alice Chen";  
19     alice.major = "Computer Science";  
20 }
```